

MCA PROJECT — TECHNICAL GUIDE

Secure AWS CI/CD Pipeline with Automated DevSecOps Security Gates

A companion technical guide covering the system architecture, implementation, verification results, and viva/interview preparation for the actual, working implementation in this repository.

Node.js / Express

AWS CodePipeline · CodeBuild · ECS Fargate

Terraform

Semgrep · Gitleaks · Checkov · Trivy · Syft

Fail-Closed Policy Engine

Author	Raghav Anand
Programme	Master of Computer Applications (MCA)
Institution	CHRIST (Deemed to be University), Bangalore, India
Contact	raghavanand033@gmail.com
Document	Project Guide (companion to the IEEE-format research paper)
Date	September 2026

Table of Contents

PART I — UNDERSTANDING THE PROJECT

1. Project Overview	3
2. Problem Being Solved	4
3. Proposed Solution	5
4. Core Features	6
5. Complete System Architecture	9
6. End-to-End Workflow	11
7. Technology Stack	12

PART II — COMPONENT DEEP DIVES

8. Frontend Deep Dive (N/A)	14
9. Backend Deep Dive	15
10. Database	17
11. AI/ML Component (N/A)	18
12. Security	19
13. Important Code Concepts	22
14. API / Data Flow	24

PART III — RUNNING & VERIFYING THE PROJECT

15. Deployment Architecture	25
16. Local Setup Guide	26
17. Production Deployment Guide	27
18. Testing	29
19. Troubleshooting	31

PART IV — REFLECTION & PRESENTATION

20. Limitations	33
21. Future Enhancements	34
22. Resume Explanation	35
23. Viva / Interview Preparation	37

This guide documents only what is actually implemented and directly verified in the repository at `/home/user/PARA_CI-CD`. Wherever a claim is a verified fact (a test run, a header inspected, a command executed), it is marked **VERIFIED**. Wherever the project has a genuine, honestly-stated limitation, it is marked accordingly. No AWS deployment has been performed as of this writing — this is stated explicitly wherever relevant rather than implied otherwise.

1 Project Overview

Secure AWS CI/CD Pipeline with Automated DevSecOps Security Gates is a complete, working DevSecOps pipeline: a small Node.js/Express REST API acts as the payload being shipped, while the actual engineering focus of the project is the pipeline that decides, automatically and without any human "Approve" click, whether that payload is allowed to reach a running AWS environment. The core deliverable is not the API — it is a fail-closed security policy engine (`security/scripts/security-gate.sh`) that reads normalized output from five independent security scanners and either lets a build proceed to Amazon ECR/ECS Fargate or stops it at the CodeBuild stage.

This is a backend + infrastructure project. There is no browser-facing frontend and no machine-learning component anywhere in the codebase — both are explicitly marked **N/A** in this guide (Sections 8 and 11) with an honest explanation of why that is the correct scope for this kind of system, rather than an oversight to be apologized for.

4 REST ENDPOINTS	5 SECURITY SCANNERS	8 TERRAFORM MODULES	4 LEAST- PRIVILEGE IAM ROLES	17 PASSING TESTS (14 UNIT + 3 INTEGRATION)	7 / 7 GATE FIXTURE SCENARIOS PASS
-------------------------------	----------------------------------	----------------------------------	--	---	---

The repository is organised into four cooperating parts:

- **The application (`app/`)** — a small, deliberately simple Express API (register, login, profile, health) that exists to give the pipeline something real to build, scan, and deploy.
- **The security-gate pipeline (`security/` , `ci/`)** — five scanner wrapper scripts, a YAML policy file, and the gate script that makes the actual pass/fail decision.
- **The infrastructure (`terraform/`)** — eight Terraform modules describing the VPC, ECS Fargate service, ALB, ECR, IAM roles, CloudWatch alarms, CodeBuild project and CodePipeline.
- **Verification (`tests/` , `app/tests/`)** — unit/integration tests for the app, and a dedicated fixture-based test harness that proves the gate's fail-closed logic is correct independent of whether any real scanner is even installed.

VERIFIED DURING THIS WRITE-UP

`npm test` → 14/14 unit tests + 3 integration tests passing; `npm run lint` clean; a full manual request flow (register → login → GET /api/users/profile) run end to end against the live app, including confirming log redaction; `bash tests/security/run-tests.sh` → "Results: 7 passed, 0 failed"; `terraform fmt -check` and `terraform validate` passed; `terraform plan` built its full resource graph and stopped only at the AWS STS credential check (no AWS credentials were available in the build sandbox).

NOT YET TRUE — STATED PLAINLY

No live AWS deployment has been performed. Terraform is verified as internally consistent and ready to `apply`, not deployed. There is a supplementary Vercel deployment path for the API alone (Section 15), which is deployable via `vercel --prod` but whose live status is not confirmed as of this writing.

2 Problem Being Solved

2.1 The gap between "we scan" and "we enforce"

Most teams that adopt security scanning in CI/CD do so in an *advisory* way: a scanner runs, a report is generated, maybe a dashboard turns red — and the deployment proceeds regardless, because nothing in the pipeline's control flow actually depends on the scan's result. This decouples **detection** from **prevention**. A perfectly accurate scanner has zero practical effect if a human has to notice the report and choose to act on it before the next deploy goes out. `docs/research-notes.md` frames this precisely as the problem this project targets: making enforcement mechanical, automated, and resistant to silent bypass, rather than "add a manual approval step and hope someone reads the report before clicking approve."

2.2 The specific failure mode this project targets: "fail open by accident"

Even pipelines that do attempt to block on scan results usually get this wrong in a specific, dangerous way: if a scanner isn't installed, crashes, times out, or produces a malformed report, most naive gate implementations either error out in a way that gets swallowed by shell scripting (`set -e` gaps, unchecked exit codes) or explicitly treat "no report" as "nothing to report" — i.e. as a pass. This is a silent, incremental way for a security control to stop functioning without anyone noticing, because the pipeline keeps going green.

2.3 Why this matters for a real deployment pipeline

- A known-CVE dependency, an unsafe code pattern, a committed secret, a misconfigured Terraform resource, or a vulnerable base image can each individually reach production if even one of five scan categories silently fails to run.
- A pipeline with a manual "Approve" gate depends entirely on a human reading a report correctly, every single time, under time pressure — which does not scale and is not auditable as a deterministic control.
- Least-privilege IAM is frequently treated as a one-time setup task rather than a design discipline applied per-actor (pipeline vs. build vs. running task), which widens blast radius when any one credential is compromised.

2.4 Problem statement, concretely

Given a CI/CD pipeline that already runs multiple security scanners, how do you make the deploy/no-deploy decision a deterministic, auditable, fail-closed function of the scan evidence — one that cannot be silently bypassed by a missing tool, a crashed scanner, a malformed report, or the absence of a human clicking a button — while still keeping the pipeline fast enough that developers don't route around it?

3 Proposed Solution

3.1 In plain terms

Every time code is pushed, the pipeline builds a Docker image and runs five different security checks against it — for insecure code patterns, vulnerable dependencies, leaked secrets, misconfigured infrastructure code, and vulnerable software inside the container image. A single script then reads all five results and compares them against a written policy: if everything is within the allowed limits, the image is pushed to the registry and the live service is updated automatically. If anything is over the limit — or if any single check didn't actually complete for any reason — the pipeline stops right there. Nothing broken or unverified ever reaches the running service, and there is no "approve anyway" button for a human to click past it.

3.2 In technical terms

The solution is a five-stage evidence pipeline feeding one deterministic policy engine:

1. **Evidence generation.** Five wrapper scripts (`security/scripts/run-*.sh`) each invoke one open-source scanner (Semgrep, npm audit/OWASP Dependency-Check, Gitleaks, Checkov, Trivy) and normalize its output into one common JSON shape: `{scanner, status, findings: {critical,high,medium,low}, details}`. Each script's own exit code communicates only whether it produced a *trustworthy report* (0 = ran, 1 = tool not installed, 2 = tool crashed) — it never makes the pass/fail decision itself.
2. **Policy definition.** `security/policy/security-policy.yaml` is the single, version-controlled source of truth for "secure enough to deploy": a severity threshold per category, plus two global fail-closed flags.
3. **Policy evaluation.** `security/scripts/security-gate.sh` loads every report, validates it, compares findings against thresholds, and produces one deterministic PASS/FAIL verdict plus a machine-readable summary — the single source of truth the rest of the pipeline depends on via its own exit code.
4. **Enforcement.** `ci/buildspec.yml` calls the gate as an ordinary shell command inside the CodeBuild `build` phase, with no error suppression. A non-zero exit fails that phase, which fails CodeBuild, which fails the CodePipeline Build stage — and CodePipeline's Deploy stage (a native ECS deploy action) has no artifact to consume if Build never finishes successfully. There is deliberately no manual Approval stage: the automated gate *is* the enforcement point.
5. **Least-privilege execution.** Every actor in this flow (the pipeline, the build, the ECS agent, the running app) has its own narrowly-scoped IAM role, so that even a full compromise of one stage cannot silently reach further than that stage's own job requires.

3.3 Why "deterministic policy engine" instead of something smarter

A compliance/security-gating decision needs to be explainable, auditable, and testable in isolation — three properties a rule-based threshold comparison has by construction and a learned model would not have without a great deal of additional engineering. Section 11 explains this design choice in full; it is treated here as the reason the "smart" part of this project is the fail-closed *semantics* of the policy engine, not a predictive model.

4 Core Features

Every feature below is implemented and present in the repository as described — endpoints, scanners, the policy engine, the fail-closed test harness, IAM separation, and digest-pinned deploys.

4.1 API endpoints

GET /health

What / Why: Liveness/readiness probe with no auth required, used by the ALB target group health check and for a fast manual smoke test.

Input: None.

Module: `app/src/routes/health.js`

How: A tiny router (`src/routes/health.js`) returns process status, uptime, and a timestamp.

Output: `200 {status, uptime, timestamp}`.

Security: Deliberately unauthenticated (health checks must not depend on credentials the health-checker doesn't have); still passes through Helmet, rate-limiting, and the redacted request logger.

POST /api/auth/register

What / Why: Creates a new user account and immediately issues a session token, so a client doesn't need a second round trip to log in.

Input: `{email, password, name}` JSON body.

Module: `src/routes/auth.js`,
`src/controllers/authController.js`

How: `express-validator` enforces a valid email, a password of at least 10 characters containing an uppercase letter, a lowercase letter and a digit, and a name of 1–100 characters; the password is hashed with `bcrypt` at 12 salt rounds before storage; a JWT is signed and returned.

Output: `201 {user:{id,email,name}, token}`;
`409` if the email is already registered; `400` on validation failure.

Security: Password never returned or logged (redacted by the logger); password hash never leaves the server; validation happens before any database/store interaction.

POST /api/auth/login

What / Why: Authenticates a user and issues a JWT for subsequent requests.

Input: {email, password}.

Module: src/controllers/authController.js ,
function login()

How: Validates the email/password shape, looks up the user, and — this is the standout security detail in the whole app — always runs `bcrypt.compare()` against either the real stored hash or a fixed dummy bcrypt hash even when no such user exists, so that a non-existent-email response and a wrong-password response take the same shape of work and cannot be distinguished by response timing. This directly prevents a user-enumeration attack via timing analysis.

Output: 200 {user, token} on success; 401 "Invalid email or password" on any failure — the same message and status whether the email doesn't exist or the password is wrong.

Security: Constant-shape response (see Section 13.4 for the exact code walkthrough); no distinction in error message between "no such user" and "wrong password".

GET /api/users/profile

What / Why: Returns the authenticated user's own profile — the one protected resource in the demo API, used to prove the JWT auth chain works end to end.

Input: No body; requires a valid Bearer token.

Module: src/routes/users.js ,
src/middleware/auth.js , src/controllers/
UserController.js

How: The `authenticate` middleware reads the `Authorization: Bearer <token>` header, verifies the JWT's signature and issuer claim with `jsonwebtoken` , and attaches the decoded payload to `req.user` before the controller ever runs.

Output: 200 {user} on success; 401 if the header is missing, malformed, or the token is invalid/expired.

Security: Signature and issuer are both checked (not just "is this valid JSON"); the raw token is never logged (redacted by both the pino-http and the app logger's redaction paths).

4.2 The five security scanners

Script	Tool	Scans	Normalized status values
run-sast.sh	Semgrep (p/security-audit + p/nodejsscan)	app/src — static code patterns	completed / tool_missing / error
run-dependency-scan.sh	npm audit, OWASP Dependency-Check	app/ dependency tree	same three values
run-secret-scan.sh	Gitleaks	Working tree + git history	same three values
run-iac-scan.sh	Checkov	terraform/	same three values
run-container-scan.sh	Trivy	The just-built Docker image	same three values

Each script's exit code is **deliberately separate from the pass/fail decision**: 0 means the scan ran and produced a report (regardless of what it found), 1 means the underlying tool isn't installed, 2 means the tool crashed mid-run. The gate — not the scanner script — decides what any of that means for deployability.

4.3 The policy engine (`security-gate.sh`)

Walked through in full in Section 13.1. In summary: loads and validates the policy YAML (hard-aborts with exit 3 on a malformed policy, since a broken policy can never be interpreted as "no policy, allow everything"); for each of five categories, checks the report exists, parses as JSON, and has `status:"completed"` — treating anything else as a policy violation unless a fail-closed flag is explicitly disabled; compares severity counts against thresholds (secrets get a separate zero-tolerance rule); prints a human-readable PASS/FAIL table and writes a machine-readable `security-gate-summary.json`; exits 0/1/3.

4.4 The fail-closed test harness

`tests/security/run-tests.sh` runs the *real* `security-gate.sh` against 7 hand-authored fixture report sets, using a `SECURITY_GATE_REPORTS_DIR` environment override so the gate reads fixtures instead of live scanner output — proving the gate's logic is correct without needing any scanner actually installed. Scenarios: clean → PASS; container critical=2 → FAIL; one secret finding → FAIL (zero tolerance); a missing report file → FAIL closed; a report containing literal text "this is not json" → FAIL closed; a report with `status:"error"` → FAIL closed; a report with `status:"tool_missing"` → FAIL closed.

VERIFIED

`bash tests/security/run-tests.sh` was run directly and produced **"Results: 7 passed, 0 failed."**

4.5 Least-privilege IAM separation

Four distinct roles (CodePipeline, CodeBuild, ECS task execution, ECS task) each trust exactly one AWS service principal and carry only the permissions that actor needs, with every unavoidable wildcard action documented inline and additionally restricted by an IAM `Condition` where AWS allows it. Fully detailed in Section 12.4.

4.6 Digest-pinned deploys

`ci/scripts/push-image.sh` resolves the just-pushed image's immutable content digest via `docker inspect` and writes that digest — not a mutable tag — into `imagedefinitions.json`, the one artifact CodePipeline's Deploy stage consumes. This closes a specific tampering window: a mutable tag could in principle be repointed between the gate's PASS decision and ECS pulling the image; a content digest cannot be repointed without also changing the bytes, which would be a different digest entirely.

5 Complete System Architecture

The system has two nested architectures worth understanding separately: the AWS/pipeline architecture (how code becomes a running service) and the Express application's own internal architecture (how one HTTP request is handled). Both are shown below as actual figures.

5.1 End-to-end system architecture

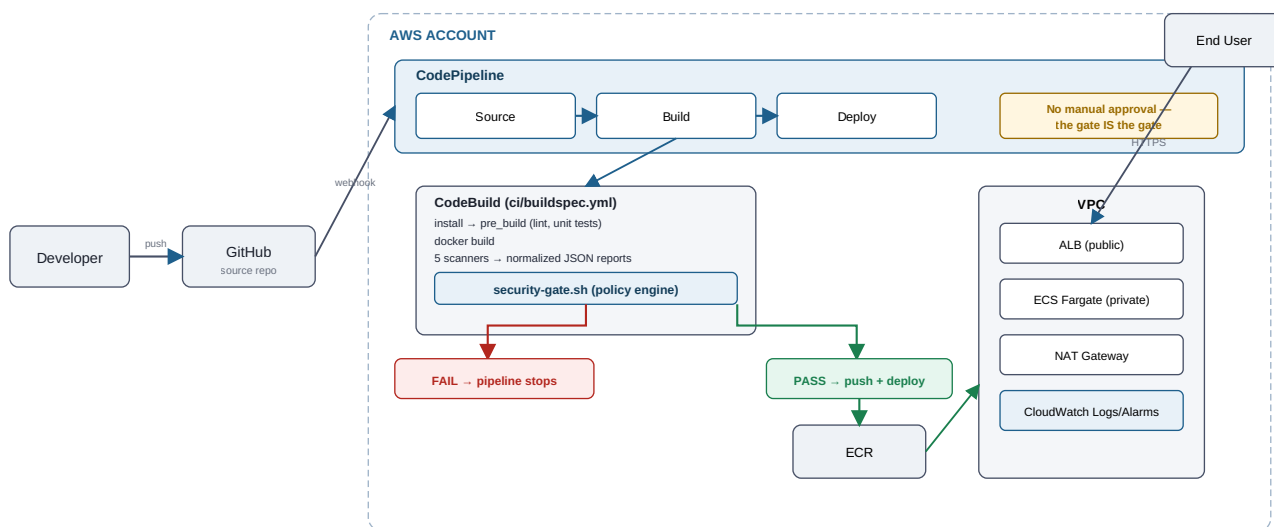


Figure 5.1 — Developer push through CodePipeline (Source → Build → Deploy) to ECS Fargate. The security gate inside the CodeBuild Build stage is the only enforcement point; there is no manual approval action.

5.2 AWS network architecture

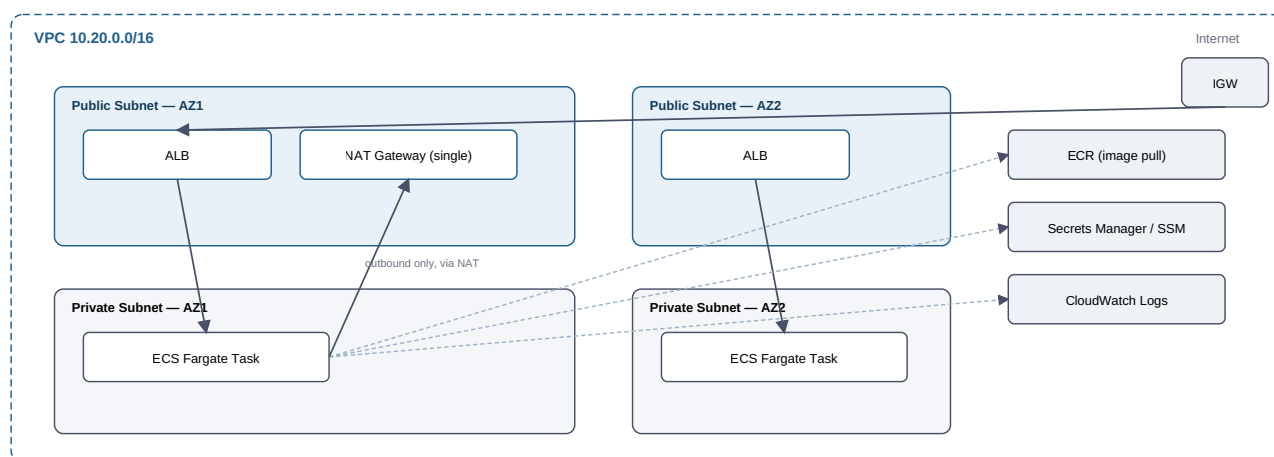


Figure 5.2 — ECS Fargate tasks run only in private subnets with no public IP; the ALB in public subnets is the sole ingress path; outbound traffic (image pulls, AWS API calls) leaves through a single, cost-optimized NAT gateway.

5.3 Express application internal request flow

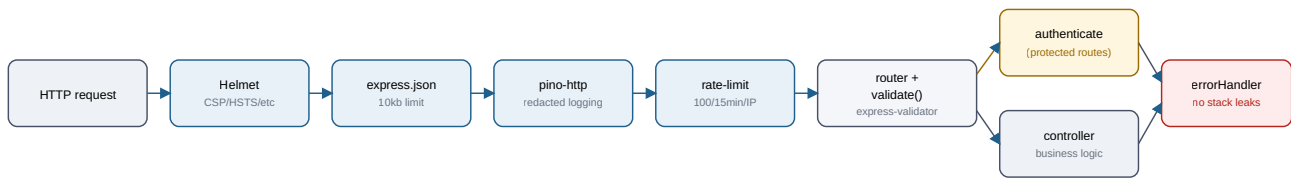


Figure 5.3 — Internal middleware stack assembled by `createApp()` in `app/src/app.js`. Every request passes through Helmet, body parsing, redacted logging and rate limiting before reaching route-level validation, optional JWT authentication, and the controller; every error path ends in the centralized handler.

6 End-to-End Workflow

6.1 Developer push → deploy-or-block decision

1. Developer commits and pushes to the configured branch (default `main`) on GitHub.
2. A CodeStar Source Connection (no stored PAT/token) notifies CodePipeline; the **Source** stage pulls the exact commit.
3. The **Build** stage starts a CodeBuild job running `ci/buildspec.yml`:
 - `install` — Node 20 setup, `npm ci`, best-effort installs of Semgrep/Checkov (pip) and Gitleaks/Trivy/Syft (install scripts), each wrapped so a failed tool install doesn't kill the build outright — it surfaces later as `tool_missing`, which the gate fails closed on with a clear reason.
 - `pre_build` — lint, unit tests, ECR login: cheap, fast checks run before any expensive Docker build or scan.
 - `build` — Docker image built; all five scanners run (each captured exit code explicitly ignored for the pass/fail decision); best-effort Syft SBOM generation; then `security-gate.sh` runs with **no** error suppression.
4. **If the gate exits non-zero**, the `build` phase fails, CodeBuild fails, the CodePipeline Build stage fails, and the pipeline stops — `post_build` /the push script never run, and the Deploy stage never executes because there's no successful Build output to consume.
5. **If the gate exits 0**, `ci/scripts/push-image.sh` tags and pushes the image to ECR, resolves its content digest, and writes `imagedefinitions.json` pinned to that digest. `post_build` prints the human-readable gate summary into the CodeBuild log (captured by CloudWatch).
6. Only `imagedefinitions.json` is published as the pipeline artifact — the exact format AWS's native ECS deploy action expects.
7. The **Deploy** stage updates the ECS service to the new, digest-pinned image. The new task revision comes up behind the ALB; the old revision is drained.

6.2 End-user HTTP request → response

1. Client sends an HTTPS request to the ALB.
2. ALB forwards to a healthy ECS Fargate task in a private subnet.
3. Inside the app: Helmet sets secure headers on the eventual response → body parsed (10kb JSON limit) → the request is logged via `pino-http` with sensitive header/body fields redacted → the global rate limiter checks the requester's IP against the configured window → the route's own `express-validator` chain validates input shape → for a protected route, the `authenticate` middleware verifies the JWT signature and issuer and attaches `req.user` → the controller executes the actual logic (e.g. hashing/ comparing a password, reading/writing the in-memory user store, signing a token) → a JSON response is sent, or an error is passed to `next(err)` and handled centrally by `errorHandler.js`, which returns a consistent JSON error shape and never leaks a stack trace to the client (stack traces go to the server-side log only).

7 Technology Stack

Technology	Purpose	Where Used	Why Used
Node.js 20 / Express 4	REST API runtime and routing	<code>app/src</code>	Fast to build a small, realistic API surface for the pipeline to build/scan/deploy; wide ecosystem of security middleware.
Docker	Containerizes the app for ECS Fargate	<code>app/Dockerfile</code> , CodeBuild build phase	ECS Fargate requires a container image; Docker is the standard build/runtime format the whole pipeline (Trivy, ECR, ECS) is built around.
Terraform $\geq$ 1.5	Infrastructure as Code for the entire AWS stack	<code>terraform/</code> (8 modules)	Declarative, reviewable, versioned infrastructure — also what Checkov scans as part of the IaC security gate.
AWS CodePipeline	Orchestrates Source → Build → Deploy	<code>terraform/modules/codepipeline</code>	Native AWS orchestration with a built-in ECS deploy action that consumes <code>imagedefinitions.json</code> directly.
AWS CodeBuild	Runs the build, tests, all 5 scans, and the gate	<code>ci/buildspec.yml</code> , <code>terraform/modules/codebuild</code>	The one place enforcement actually happens; privileged mode gives it Docker for image builds.
Amazon ECR	Immutable, scan-on-push container registry	<code>terraform/modules/ecr</code>	Immutable tags + built-in scan-on-push are a second, independent layer beneath the pipeline's own Trivy scan.
Amazon ECS Fargate	Runs the container with no server management	<code>terraform/modules/ecs</code>	Serverless container hosting fits a small demo/student-budget deployment and removes host patching entirely.
Amazon CloudWatch	Logs, alarms, dashboards	<code>terraform/modules/cloudwatch</code>	Native, low-friction observability for task health, ALB 5xx rate, and p95 latency, notifying via SNS.
AWS IAM	Least-privilege identity for every pipeline/runtime actor	<code>terraform/modules/iam</code>	Four scoped roles instead of one broad role — limits blast radius of any single compromise (Section 12.4).
AWS Secrets Manager / SSM Parameter Store	Runtime secret and config injection	<code>terraform/modules/security</code> , ECS task definition	Keeps <code>JWT_SECRET</code> and config out of source control and out of the image entirely.
Semgrep	SAST — static code pattern scanning	<code>security/scripts/run-sast.sh</code>	Rule-based, fast, open-source, well-suited to catching known-bad patterns (e.g. command injection) in a small codebase.
Gitleaks	Secret scanning (working tree + git history)	<code>security/scripts/run-secret-scan.sh</code>	Purpose-built for exactly one job — leaked-credential detection — done well and fast.
Checkov	IaC misconfiguration scanning	<code>security/scripts/run-iac-scan.sh</code>	Broad, actively maintained Terraform rule coverage (open SGs, public buckets, missing encryption, etc.).

Trivy	Container image vulnerability scanning	<code>security/scripts/run-container-scan.sh</code>	Scans OS packages and language dependencies baked into the built image, complementing ECR's own scan-on-push.
Syft	SBOM generation (CycloneDX + SPDX)	<code>ci/scripts/generate-sbom.sh</code>	Produces a software bill of materials for the built image; currently informational, not a gate input (see Section 12.5).
Jest / Supertest	Unit and integration testing for the API	<code>app/tests</code> , <code>tests/integration</code>	Standard Node.js testing stack; Supertest exercises the Express app in-process without a real network socket.
jq / yq	JSON/YAML parsing inside all bash scanner and gate scripts	<code>security/scripts</code>	Lets the gate and scanner wrappers be portable shell scripts rather than requiring a heavier runtime just to parse policy/report files.
Vercel (supplementary)	Standalone hosting for the API only, bypassing the pipeline entirely	<code>app/api/index.js</code> , <code>app/vercel.json</code>	Lets the API be poked at without first standing up AWS infrastructure; explicitly does not exercise the security-gate pipeline (Section 15.2).

8 Frontend Deep Dive

N/A — FUTURE ENHANCEMENT

This project has no browser-facing frontend or client application, and that is a deliberate scope decision rather than an unfinished piece. The engineering focus of this project is the CI/CD pipeline and its automated security-gating logic — the Express API in `app/` exists specifically to give that pipeline something real to build, test, scan, and deploy, not to be a product with a user interface. Building a UI would have added surface area (a separate build/deploy path, its own dependency tree, its own attack surface) without adding anything to the actual research/engineering question this project answers: *how do you make a security gate enforce automatically and fail closed?* A frontend (e.g. a small React dashboard visualizing gate results, or a login UI consuming the existing auth endpoints) is a reasonable future enhancement (see Section 21) precisely because the API underneath it already exists and is stable — but it was correctly left out of this project's actual scope.

9 Backend Deep Dive

9.1 Application factory and composition root

`app/src/app.js` exports a single `createApp()` factory function that assembles the entire middleware stack and mounts the three routers (`health`, `auth`, `users`) before returning the Express `app` instance. Keeping this in a factory rather than a top-level script is what makes the same application runnable three different ways without duplicating any wiring: `src/server.js` calls `createApp().listen()` for local/ECS use; `app/api/index.js` calls the same factory and exports it as a Vercel serverless handler; and Supertest in the test suite calls it directly, in-process, with no network socket at all.

Middleware stack, in the exact order applied

1. `helmet()` — sets Content-Security-Policy, Strict-Transport-Security, X-Frame-Options, and the rest of Helmet's default secure-header set.
2. `express.json({limit:'10kb'})` — parses JSON bodies, capped at 10kb to blunt trivial body-size DoS attempts.
3. `pino-http` — structured JSON request logging, configured with `redact: ['req.headers.authorization', 'req.headers.cookie']` at the HTTP-log level, layered on top of the logger's own field-level redaction (Section 12.2).
4. `express-rate-limit` — 100 requests / 15 minutes per IP by default (both configurable via env), standard rate-limit headers enabled.
5. Route mounting: `/health`, `/api/auth`, `/api/users`.
6. `notFoundHandler` then `errorHandler` — the two terminal middlewares; every request that falls through routing or throws ends up here.

VERIFIED DIRECTLY

The app was run and its response headers inspected directly. The response included `content-security-policy`, `strict-transport-security`, and `x-frame-options: SAMEORIGIN` among Helmet's default headers, confirming Helmet is actually wired in and not just imported.

9.2 Configuration (`src/config/index.js`)

All configuration is env-driven via `dotenv`. A small `required(name, fallback)` helper enforces fail-fast behavior for secrets: outside of `NODE_ENV=test`, a missing `JWT_SECRET` throws at startup rather than allowing the app to boot with an undefined or empty signing key. In test mode, a safe fallback (`test-only-insecure-secret-do-not-use-in-prod`) is used so the suite never needs a real secret. Every other setting (port, rate-limit window/max, bcrypt salt rounds, JWT expiry/issuer, log level) has a sane default and an env override.

9.3 Error hierarchy (`src/utils/errors.js`)

A small class hierarchy — `ValidationError`, `AuthenticationError`, `ConflictError` — gives each error type a fixed HTTP status code, so controllers can `throw new ConflictError(...)` and trust that the centralized handler will map it to `409` consistently, rather than every controller re-deciding status codes ad hoc.

9.4 Centralized error handling (`src/middleware/errorHandler.js`)

Every error, from any controller or middleware, funnels through one handler that returns a consistent `{error: {message, code}}` JSON shape and an appropriate status code. Critically, it never returns a raw stack trace or error object to the client — stack traces are logged server-side only, via the same redacting logger used everywhere else.

9.5 Validation middleware (`src/middleware/validate.js`)

Wraps an array of `express-validator` chains, runs them, and — if any fail — throws a `ValidationError` with the collected messages before the request ever reaches a controller. This keeps input validation entirely declarative at the route level (see `src/routes/auth.js`'s `registerValidation` / `loginValidation` arrays) rather than scattered through controller bodies.

9.6 API reference table

Method	Endpoint	Purpose	Input	Output	Auth
GET	<code>/health</code>	Liveness/readiness check	—	200 {status, uptime, timestamp}	None
POST	<code>/api/auth/register</code>	Create account, issue token	{email, password, name}	201 {user, token} / 409 / 400	None
POST	<code>/api/auth/login</code>	Authenticate, issue token	{email, password}	200 {user, token} / 401	None
GET	<code>/api/users/profile</code>	Return caller's own profile	—	200 {user} / 401	Bearer JWT

VERIFIED DIRECTLY

The full flow — register, login with the returned credentials, then `GET /api/users/profile` using the JWT from login — was run end to end against the running app and succeeded, including confirming that a request carrying an `Authorization` header was logged as `"authorization": "[Redacted]"`, never the raw token.

10 Database

KNOWN LIMITATION, STATED HONESTLY

There is **no persistent database** in the current implementation. `app/src/models/userStore.js` is an in-memory JavaScript `Map` keyed by user id, with a secondary lookup by email — its own header comment in the source states plainly that a real deployment would back this with RDS or DynamoDB instead. This is a documented simplification made to keep the project's scope on the pipeline/security-gate work, not an accidental gap discovered late.

Practical consequences of this choice, stated directly:

- State does not survive a process restart. On ECS Fargate, one long-lived task process holds the data for its lifetime; a task replacement (deploy, crash, scale event) clears it.
- On the supplementary Vercel deployment path, each serverless invocation may run on a fresh, independently cold-started instance — a user registered in one invocation is not guaranteed to be found by `login` in the next (Section 15.2).
- There is no horizontal scaling story for user data as implemented — running two ECS tasks would give each task its own, inconsistent view of registered users.

10.1 The record shape actually used

This is not a formal schema or ER diagram — there is no relational structure to model, only a flat record shape held in memory:

Field	Type	Set by	Notes
<code>id</code>	UUID string	<code>crypto.randomUUID()</code> at registration	Primary key of the <code>Map</code> .
<code>email</code>	string	Client-supplied, validated + normalized	Secondary lookup index used by <code>findByEmail()</code> .
<code>name</code>	string	Client-supplied, validated (1–100 chars)	Returned in API responses, never used for auth.
<code>passwordHash</code>	bcrypt hash string	<code>bcrypt.hash(password, 12)</code>	Never returned in any API response.
<code>createdAt</code>	ISO-8601 timestamp	<code>new Date().toISOString()</code>	Set once at registration.

A production version of this project would replace `userStore.js`'s in-memory `Map` with a thin repository interface backed by DynamoDB (single-table, `PK=USER#<id>`, GSI on email) or RDS/Postgres, without needing to change the controller layer above it, since the store is already accessed through a small, narrow function interface (`findByEmail`, `create`).

11 AI/ML Component

N/A — DELIBERATE DESIGN CHOICE, NOT A GAP

There is no machine learning anywhere in this project, and that absence is the correct engineering choice for what this system actually decides. The security gate's job is to make a binary, high-stakes, compliance-relevant decision — *does this build reach production or not* — from a small set of well-defined numeric inputs (severity counts per category). For a decision of that shape, a deterministic, rule-based policy engine is strictly preferable to a learned model, for reasons that matter in a real security context, not just convenience:

- **Auditability.** Every FAIL the gate produces traces to one specific, human-readable rule (`critical 2 > 0`) that a security reviewer can verify by reading the policy YAML — no model weights or training data to interrogate.
- **Determinism and testability.** The exact same report, run through the gate a thousand times, produces the exact same verdict — which is precisely what makes the 7-scenario fixture test suite (Section 4.4, Section 18.3) a meaningful, complete correctness proof rather than a statistical sample.
- **No training-data dependency or drift.** A learned classifier over "is this build safe to ship" would need labeled historical data that doesn't exist for a new project, and would be vulnerable to distribution shift as the codebase and dependency set evolve — a threshold-based policy has neither problem.
- **Regulatory/compliance fit.** Security-gating decisions in real organizations typically need to be explainable to an auditor in plain terms; "the policy says `critical=0` and Trivy found 2" is that explanation, and it is the only explanation this design ever needs to give.

This is why Section 3.3 frames the actual "smart" engineering in this project as the fail-closed *semantics* of the policy engine — correctly handling missing/malformed/crashed evidence as a violation rather than a skip — not a predictive model. An ML-based anomaly-detection layer over historical scan data (e.g. flagging an unusual spike in findings for a given commit) is a plausible, legitimate future enhancement on top of this system (Section 21), but it would be an addition to the deterministic gate, never a replacement for it.

12 Security

12.1 Authentication and password handling

- **bcrypt at 12 salt rounds** for password hashing (`config.bcrypt.saltRounds`), a widely-accepted adaptive-cost factor for 2026-era hardware.
- **JWTs** signed with `JWT_SECRET`, carrying an `issuer` claim and an expiry (`JWT_EXPIRES_IN`, default 1h); verification checks both the signature and the issuer, not just that the token parses.
- **Constant-shape login response** (Section 13.4) — the single most interesting security detail in the app code, and worth being able to explain precisely in an interview.

12.2 Secrets handling and log redaction

`app/src/utils/logger.js` configures `pino` with an explicit `redact.paths` list covering `req.headers.authorization`, `req.headers.cookie`, `req.body.password`, `req.body.confirmPassword`, `req.body.token`, and wildcard patterns `*.password`, `*.token`, `*.jwt`, `*.secret`, `*.authorization`, censored to the literal string `[REDACTED]`. This was verified directly, not just read in source: a request carrying a real `Authorization` header was logged with the value replaced by the redaction marker.

In AWS, `JWT_SECRET` is generated by Terraform (`random_password`), stored in Secrets Manager, and resolved by the ECS task definition's `secrets` block at container startup — it is never hardcoded, never checked into source control, and never touches application code directly (Section 12.4 explains exactly which IAM role makes that resolution possible).

12.3 Transport and application-layer hardening

Control	Mechanism	Verified how
Secure headers	Helmet (CSP, HSTS, X-Frame-Options, etc.)	Direct header inspection on a live response
Rate limiting	<code>express-rate-limit</code> , 100 req/15 min/IP (configurable)	Read in <code>app.js</code> ; standard headers enabled
Body size limit	<code>express.json({limit: '10kb'})</code>	Read in <code>app.js</code>
Input validation	<code>express-validator</code> chains per route, enforced before controller logic runs	Read in <code>routes/auth.js</code>
Error response hygiene	Centralized handler never returns stack traces to clients	Read in <code>middleware/errorHandler.js</code>

12.4 IAM least-privilege — a real case study

Four roles, each trusting exactly one AWS service principal, each scoped to only what that actor needs:

1. CodePipeline role (trusts `codepipeline.amazonaws.com`)

- S3 read/write scoped to one artifact bucket ARN; KMS decrypt/encrypt on one CMK.
- `codebuild:StartBuild` / `BatchGetBuilds` scoped to `project/${name_prefix}-*`.
- `codestar-connections:UseConnection` for the tokenless GitHub link.

- ECS describe/register/update, scoped narrowly to this project's resources.
- `iam:PassRole` restricted to role ARNs matching `${name_prefix}-ecs-*` **and** conditioned on `iam:PassedToService = ecs-tasks.amazonaws.com` — a genuine privilege-escalation guardrail: without this condition, a compromised pipeline role could pass an over-privileged role to a service it was never meant to reach.

2. CodeBuild role (trusts `codebuild.amazonaws.com`)

- CloudWatch Logs write scoped to `/codebuild/${name_prefix}*`; same artifact-bucket/KMS pattern as the pipeline role.
- `ecr:GetAuthorizationToken` with `Resource: "*"` — a hard AWS API limitation (this action cannot be scoped to one repository); the code isolates this into its own statement with an inline comment explaining exactly why, and everything else (`PutImage`, layer upload actions) IS scoped to one ECR repository ARN.
- `secretsmanager:GetSecretValue` scoped to one JWT secret ARN only — used by integration-style tests that exercise the real auth flow against a real-shaped secret rather than a hardcoded dummy value.

3. ECS task *execution* role (trusts `ecs-tasks.amazonaws.com`) — used by the ECS agent, not application code

- ECR pull-only — no push permissions at all, in direct contrast with the CodeBuild role, which has both.
- CloudWatch Logs write scoped narrowly to `/ecs/${name_prefix}*`.
- `secretsmanager:GetSecretValue` + `ssm:GetParameters`, each scoped to specific ARNs — this is the literal mechanism that injects `JWT_SECRET` into the running container via the ECS task definition's `secrets` block, without the app or its code ever touching Secrets Manager directly and without the secret ever appearing in source control.

4. ECS task role (trusts `ecs-tasks.amazonaws.com`) — assumed by the running application itself

Deliberately near-empty, because this demo app makes no AWS API calls of its own: exactly one statement, `cloudwatch:PutMetricData`. That action cannot be ARN-scoped by AWS, so it is tightened instead with a `Condition: cloudwatch:namespace = ${name_prefix}/app`, restricting even this unavoidable wildcard action to the app's own metric namespace.

WHY THE EXECUTION-ROLE / TASK-ROLE SPLIT MATTERS

These are commonly confused. The **execution role** is what the ECS *agent* uses before your code even starts — to pull the image and resolve secrets/config into environment variables. The **task role** is what your *running application code* assumes to call AWS APIs itself. Splitting them means a compromise of the running application (task role) cannot reach the broader pull/secrets-resolution permissions the execution role has — the app's own blast radius is capped at exactly `cloudwatch:PutMetricData` on one namespace.

No role anywhere in this project has `AdministratorAccess` or an unnecessary wildcard; every unavoidable `Resource: "*" (twice: ecr:GetAuthorizationToken, cloudwatch:PutMetricData)` is documented inline and, where AWS allows it, further restricted with a `Condition`.

12.5 Threat model summary

`docs/threat-model.md` is a full STRIDE analysis (14 threats, plus a dedicated supply-chain section) mapped directly to implemented controls. A representative sample:

Threat	Control	Residual risk (stated honestly)
Vulnerable dependency exploited	npm audit + OWASP Dependency-Check every build; gate blocks CRITICAL/HIGH	A zero-day before any CVE/advisory exists is not caught until the next scan.
Leaked credential in source/history	Gitleaks scans full history + working tree; zero-tolerance gate rule	A secret already exfiltrated before detection still needs manual rotation — scanning alone doesn't rotate it.
Artifact tampering between Build and Deploy	Immutable ECR tags; digest-pinned <code>imagedefinitions.json</code>	Limited to an attacker who already has write access to the artifact bucket or ECR — which IAM restricts to the pipeline/build roles specifically.
Pipeline bypass (editing buildspec/ policy to weaken the gate)	Both files are version-controlled and go through the same PR process as app code; a GitHub Actions mirror workflow independently re-runs the same gate on every PR	A sufficiently privileged actor could still edit both in the same PR — code review, not a technical control, is the real mitigation here.
Insider threat / compromised build environment	Least-privilege IAM limits blast radius per actor	CloudTrail/Config are not deployed in this demo — the single largest documented gap versus a production-grade deployment.

12.6 Recommended future security improvements

Stated as genuine gaps, not implemented features described optimistically:

- **DAST / penetration testing** against the live endpoint — nothing in this project exercises the running service dynamically; all scanning today is static/build-time.
- **Image signing / SLSA provenance attestation** (e.g. Sigstore/cosign) — images are digest-pinned but not cryptographically signed; this is the most significant supply-chain gap noted in `threat-model.md`.
- **Automated secret rotation** for `JWT_SECRET` — currently a manual `aws secretsmanager put-secret-value` + forced ECS redeploy (Section 19), not a scheduled Lambda rotation.
- **CloudTrail + AWS Config** — not deployed in this demo (kept out of scope to keep AWS cost minimal for a student account), which is the biggest gap for insider-threat/audit-trail coverage.
- **SBOM as a gate input, not just informational** — Syft's output is generated and stored today but not yet compared against a policy the way the other five categories are.

13 Important Code Concepts

13.1 `security-gate.sh` 's decision logic, in detail

This is the single most important file in the project for a viva. Its control flow, exactly as implemented:

1. **Tool availability check.** Confirms `jq` and `yq` are on `PATH`; if not, `exit 3` immediately — the gate cannot even parse its own policy without them.
2. **yq variant detection.** Two unrelated tools are both called `yq` (Go/mikefarah, which needs `-o=json`; Python/kislyuk, which emits JSON by default). The script probes `yq --help` for the `-o, --output-format` flag and picks the right invocation, so either variant works transparently.
3. **Policy validation.** Parses the YAML to JSON and checks all six required top-level keys exist (`version` , `sast` , `dependencies` , `secrets` , `iac` , `container` , `gate`) via one `jq -e` boolean expression. Any failure here — bad YAML or a missing section — is `exit 3` , distinct in kind from a policy *violation*: a broken policy file means the gate literally cannot run, and must never be silently interpreted as "no policy, so allow everything."
4. **Per-category evaluation.** `load_report()` checks, in order: does the report file exist; does it parse as valid JSON (`jq -e .`); is `status == "completed"` ; do all four findings counts exist. Any failure sets `REPORT_VALID=false` with a specific reason string. Back in the caller, that reason is checked: if it mentions "status is" (i.e. the scanner ran but reported `tool_missing / error`), the `fail_closed_on_scanner_crash` flag governs; otherwise (missing file, bad JSON) `fail_closed_on_missing_report` governs. With both flags at their default `true` , every one of these becomes a **FAIL**, not a silent skip.
5. **Threshold comparison.** For a valid report, each severity count is compared against its policy threshold; `-1` means "unenforced". Any count exceeding its threshold is collected as a violation string (e.g. `critical 2>0`) and the category is marked FAIL with the exact numbers shown.
6. **Secrets' own rule.** `evaluate_secrets()` does not use severity thresholds at all — any finding count `>0` is an unconditional FAIL when `block_on_any_finding: true` (the default), reflecting that a single leaked credential is unconditionally unacceptable regardless of how the scanner classified its severity.
7. **Aggregation and output.** A boolean `GATE_PASS` starts `true` and flips to `false` the moment any category fails. A human-readable table titled "**SECURITY GATE RESULT**" is printed and saved, alongside a machine-readable `security-gate-summary.json` .
8. **Final exit.** `exit 0` only if every enabled category passed; otherwise `exit 1` . This exit code — not the individual scanner exit codes, which `ci/buildspec.yml` deliberately captures and ignores for the deploy decision — is the single value the rest of the pipeline depends on.

13.2 The scanner-output-normalization pattern

Five completely different tools (Semgrep, npm audit/OWASP DC, Gitleaks, Checkov, Trivy) have five completely different native output formats. Rather than teaching the gate about all five formats, every wrapper script under `security/scripts/run-*.sh` translates its tool's output into one shared JSON shape before the gate ever sees it:

```
{
  "scanner": "trivy",
  "status": "completed | tool_missing | error",
  "generated_at": "2026-01-01T00:00:00Z",
```

```
"findings": { "critical": 0, "high": 0, "medium": 0, "low": 0 },
"details": [ /* up to 50 individual findings */ ]
}
```

This is a classic **adapter pattern** applied at the shell-script level: adding a sixth scanner tomorrow means writing one new wrapper that emits this shape — `security-gate.sh` itself needs zero changes. It is also what makes the fixture-based test harness (Section 4.4) possible at all: fixtures only need to match this one normalized shape, not simulate five different real tools.

13.3 The execution-role vs. task-role IAM split

Covered in depth in Section 12.4; restated concisely because it is a frequently-confused AWS concept worth being able to explain crisply: the **execution role** is assumed by the ECS agent to pull the image and resolve secrets/config *before* your container starts; the **task role** is assumed by your running application code to call AWS APIs *while* it runs. They are separate IAM roles specifically so that a compromise of the running application (which only has the task role) cannot reach the broader pull/secrets-resolution permissions that only the execution role holds.

13.4 The constant-shape login timing-attack mitigation

The exact code, from `app/src/controllers/authController.js` :

```
const dummyHash = '$2a$12$CwTycUXWue0Thq9StjUM0uJ8yqYXt5rQmUxRlqrRtG3jsBb.i0K.C';
const hashToCompare = user ? user.passwordHash : dummyHash;
const passwordMatches = await bcrypt.compare(password, hashToCompare);

if (!user || !passwordMatches) {
  throw new AuthenticationError('Invalid email or password');
}
```

Without this, a naive implementation would short-circuit on "user not found" and return immediately — skipping the (deliberately slow, adaptive-cost) `bcrypt.compare()` call entirely for non-existent emails, while a real, registered email always pays that cost. An attacker measuring response times across many requests could use that timing difference to enumerate which emails are registered, entirely without guessing any password. By always running `bcrypt.compare()` — against the real hash when the user exists, and against a fixed, pre-computed dummy hash when it doesn't — both code paths do the same category and magnitude of work, and the error message returned is identical either way ("Invalid email or password"), removing both the timing and the message as a distinguishing signal.

14 API / Data Flow

14.1 Request flow through the Express middleware stack

Diagrammed in Figure 5.3. In text form:
 HTTP request → Helmet (adds secure headers to the response) → express.json (parses/limits body) → pino-http (structured, redacted request log) → rate limiter (per-IP window check) → router dispatch → route-level express-validator chain → [authenticate, for protected routes] → controller → response, or errorHandler on any thrown/passed error.

14.2 Artifact flow through CodePipeline stages

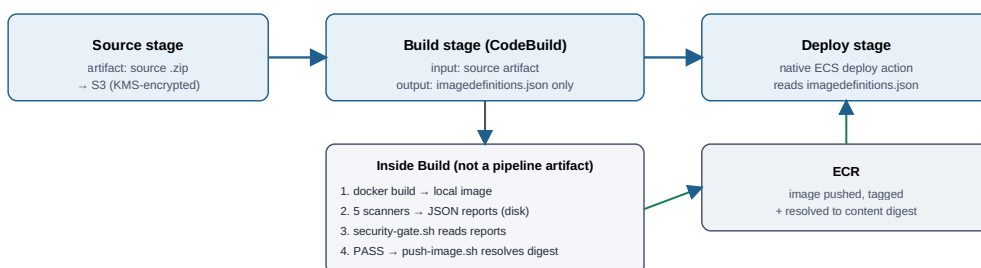


Figure 14.1 — Only `imagedefinitions.json` (digest-pinned, not tag-pinned) crosses the Build → Deploy stage boundary as a pipeline artifact; scan reports and the gate summary live only in the CodeBuild log/CloudWatch.

The reason only one small JSON file is published as the artifact (rather than also publishing the five scan reports and the gate summary) is a deliberate simplification that keeps this buildspec's single Build action compatible with the single-output-artifact CodePipeline stage shape — the full reports and PASS/FAIL summary are still fully captured, just via the CodeBuild CloudWatch log group (`post_build` prints the summary explicitly) rather than as a second pipeline artifact.

15 Deployment Architecture

15.1 Local development

```
cd app && npm install && npm run dev
```

Reads configuration from `.env` (copied from `.env.example` — the real `.env` is never committed). `npm run dev` runs `node --watch src/server.js`, restarting on file changes without a separate nodemon dependency.

15.2 Real AWS path — the one this project is actually about

Push to GitHub → CodeStarSourceConnection triggers CodePipeline → CodeBuild runs the full install/pre_build/build/post_build flow, including the security gate → on PASS, image is pushed to ECR and the ECS service is updated → on FAIL, the pipeline stops at Build and nothing deploys.

STATED PLAINLY

This AWS path requires the user to provision it themselves via `terraform apply` (Section 17). **It has not been deployed as of this writing.** No live AWS environment exists for this project; Terraform has been verified as internally consistent (`fmt`, `validate`, a successful `plan` graph build) but never actually applied against a real AWS account.

15.3 Supplementary path — Vercel (API only, not the pipeline)

`app/api/index.js` wraps the same `createApp()` factory as a Vercel serverless function; `app/vercel.json` handles routing and sets the runtime env vars the app's config layer requires. This lets the API alone run on Vercel — explicitly **not** exercising the security-gate pipeline: Vercel's build does not run Semgrep, Trivy, Checkov, or `security-gate.sh` at all, and there is no CodePipeline/CodeBuild/ECS/Terraform involved in this path whatsoever.

REAL LIMITATION OF THIS PATH

The in-memory user store does not reliably persist across Vercel's serverless cold starts, unlike a long-lived ECS task — a user registered in one invocation may not be found by `login` in a different, freshly cold-started invocation.

This path is deployable via `vercel --prod` from `app/`; its live deployment status is not confirmed as of this document's writing, and no URL is asserted here.

16 Local Setup Guide

Exact commands to run the API locally:

```
cd app
cp ../.env.example .env      # edit JWT_SECRET etc. for local use
npm install
npm run dev                  # starts on http://localhost:3000
curl http://localhost:3000/health
```

Expected output from the health check: a JSON body with `status`, `uptime`, and `timestamp` fields — this is exactly what was verified directly during this document's preparation.

To exercise the full auth flow locally:

```
curl -X POST http://localhost:3000/api/auth/register \
-H "Content-Type: application/json" \
-d '{"email":"demo@example.com","password":"Passw0rd123","name":"Demo User"}'

curl -X POST http://localhost:3000/api/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"demo@example.com","password":"Passw0rd123"}'

# copy the "token" field from the login response, then:
curl http://localhost:3000/api/users/profile \
-H "Authorization: Bearer <token>"
```

17 Production Deployment Guide

17.1 AWS path (Terraform) — documented, verified as internally consistent, not yet executed

Per `docs/deployment.md`, in order:

1. **Prerequisites:** an AWS account (student/free-tier is sufficient), AWS CLI v2 with permissions to create IAM/VPC/ECS/ECR/CodePipeline/CodeBuild/KMS/Secrets Manager/S3 resources, Terraform $\geq 1.5.0$, Docker, a GitHub fork, Node.js ≥ 20 .
2. **GitHub + CodeStar Connection:** push the repo to your own GitHub account; in the AWS Console create a CodeStar/CodeConnections connection to GitHub and complete the interactive OAuth handshake (the one step Terraform cannot automate); copy the resulting connection ARN into `terraform.tfvars`.
3. **Terraform state backend (one-time, manual):** create an S3 bucket (versioned, encrypted) and a DynamoDB lock table by hand, since the backend cannot create its own storage.
4. **Terraform apply:**

```
cd terraform/environments/dev
cp backend.tf.example backend.tf          # set bucket, dynamodb_table, region
cp terraform.tfvars.example terraform.tfvars # set github_owner, github_repo,
codestar_connection_arn
terraform init
terraform plan -var-file=terraform.tfvars
terraform apply -var-file=terraform.tfvars
```

This provisions, in order: KMS key + Secrets Manager/SSM entries, VPC/subnets/security groups, ECR repository, S3 artifact bucket, all four IAM roles, the ECS cluster/ALB/service (seeded with a placeholder image tag), CloudWatch log groups/alarms, the CodeBuild project, and finally CodePipeline itself.

5. **Trigger the pipeline:** it fires automatically on the next push to the configured branch, or manually via `aws codepipeline start-pipeline-execution --name $(terraform output -raw codepipeline_name)`.

STATED PLAINLY, AGAIN

`terraform apply` has **not been run** against a real AWS account for this project. What was verified: `terraform fmt -check` and `terraform validate` both passed, and `terraform plan` successfully built the complete resource graph — it stopped only at the AWS STS credentials check, because no real AWS account/credentials were available in the build sandbox this project was verified in. This is Terraform confirmed as internally consistent and ready to deploy, not a deployed system.

Cost notes (from `docs/deployment.md`)

The stack is deliberately sized to be cheap for a student account: a single 0.25 vCPU / 512 MB Fargate task, a single NAT gateway (not per-AZ — the single largest recurring cost line item), an ECR lifecycle policy capping retained image history, and no ACM certificate/Route 53 zone (the ALB uses plain HTTP). The guide explicitly recommends destroying the stack between demo sessions to avoid accruing NAT gateway charges.

17.2 Supplementary path — Vercel (API only)

```
npm install -g vercel    # one-time
cd app
```

```
vercel login          # one-time browser auth
vercel --prod
```

Accept CLI defaults (project root `./`, Node.js auto-detected). This is deployable via `vercel --prod` from `app/`; deployment status is not confirmed as of this document's writing, and this path never runs the security-gate pipeline (Section 15.3).

18 Testing

18.1 Application tests

```
cd app
npm test      # jest --coverage
npm run lint # eslint src tests
```

VERIFIED DIRECTLY DURING THIS WRITE-UP

`npm test` → **14/14 unit tests passing** (`app/tests/unit/{auth,health,validation}.test.js`) plus **3 passing integration tests** (`tests/integration/api.integration.test.js` , exercising the real Express app in-process via Supertest — register → login → protected profile fetch, and negative cases). `npm run lint` completed clean with no ESLint errors.

18.2 What the tests actually cover

- **Unit — health:** the `/health` endpoint returns the expected shape and status code.
- **Unit — auth:** registration validation (email format, password complexity, name length), duplicate-email conflict handling, login success/failure paths, and JWT issuance shape.
- **Unit — validation:** the `validate()` middleware correctly short-circuits to a `ValidationError` on malformed input before reaching a controller.
- **Integration:** the full register → login → authenticated-profile-fetch flow against the real, assembled Express app (via `createApp()` and Supertest), not against mocked internals.

18.3 The 7-scenario security gate fixture suite

```
bash tests/security/run-tests.sh
```

#	Fixture scenario	Expected gate result
1	All five categories clean/within threshold	PASS
2	Container scan report with <code>critical: 2</code> (threshold is 0)	FAIL — threshold violated
3	Secrets report with 1 finding	FAIL — zero tolerance
4	<code>secrets/secret-scan-report.json</code> missing entirely	FAIL — closed (missing report)
5	An IaC report file containing the literal text "this is not json"	FAIL — closed (invalid JSON, caught by <code>jq -e .</code>)
6	A report with <code>status:"error"</code> (simulated scanner crash)	FAIL — closed
7	A report with <code>status:"tool_missing"</code> (simulated missing tool)	FAIL — closed

VERIFIED DIRECTLY

`bash tests/security/run-tests.sh` was run against the real `security-gate.sh` and produced **"Results: 7 passed, 0 failed."** This runs the actual gate script (not a reimplementaion of its logic) against hand-authored

fixtures via a `SECURITY_GATE_REPORTS_DIR` environment override, so it proves gate correctness independent of whether any real scanner is even installed.

18.4 Terraform verification

```
cd terraform
terraform fmt -check
terraform validate
terraform plan -var-file=<your tfvars> # builds the graph; stops at STS auth without real
credentials
```

`terraform fmt -check` and `terraform validate` both passed. `terraform plan` built the complete resource graph successfully; it did not complete because no real AWS credentials were available in the verification sandbox, which is expected and does not indicate a configuration problem.

19 Troubleshooting

Selected, real content from `docs/troubleshooting.md` :

19.1 Installing the security scanners locally

Every `security/scripts/run-*.sh` script fails loudly (writes a `status:"tool_missing"` report and exits 1) rather than silently skipping itself when its tool isn't installed — and the gate fails closed on that. To get real results locally:

```
pip install semgrep          # SAST
brew install dependency-check # deeper SCA than npm audit alone (optional)
brew install gitleaks        # secret scanning
pip install checkov          # IaC scanning
brew install trivy           # container scanning
curl -sSfL https://raw.githubusercontent.com/anchore/syft/main/install.sh | sh -s -- -b /usr/
local/bin # SBOM, optional
```

`npm audit` needs no separate install — it ships with npm.

19.2 `security-gate.sh` exits 3 immediately

Exit 3 means the gate itself could not run. Check, in order: are `jq` and `yq` installed; does `security/policy/security-policy.yaml` parse as valid YAML; does it have all six required top-level sections. This is by design — a broken policy file must never be interpreted as "no policy, so allow everything."

19.3 Every category shows missing/FAIL

The gate only *reads* reports — it does not generate them. Run the five scanner scripts first (`make security` runs four of them; container scanning needs a built image, so it's separate via `make docker-scan`), then run the gate.

19.4 `yq` behaves differently / unknown flag

Two unrelated tools are both called `yq`: mikefarah's Go version (needs `-o=json`) and kislyuk's Python version (JSON by default, wraps `jq`). `security-gate.sh` auto-detects which is on `PATH`. This project was built and tested against the Python (kislyuk) variant — `pip install yq`.

19.5 Docker build fails / Docker not found

The Dockerfile requires a working Docker daemon. On a machine without Docker access, `make test`, `make lint`, and `make security` (non-container scanners) still work. CodeBuild always has Docker available via `privileged_mode = true`, so the AWS pipeline build is unaffected by local Docker availability.

19.6 `terraform validate` fails with a provider download error

If `registry.terraform.io` is unreachable, download the provider zips directly from `releases.hashicorp.com` and point Terraform at a local filesystem mirror via a `provider_installation` block in `~/.terraformrc`. This is exactly how this repository's own `terraform fmt / validate / plan` were

exercised — the registry was network-blocked in the sandbox, but direct HTTPS downloads from `releases.hashicorp.com` were not.

19.7 Common environment/config mistakes

- Missing `JWT_SECRET` outside test mode — the app fails fast at startup by design (`config/index.js` 's `required()` helper), rather than booting with an undefined signing key.
- ECS tasks not stabilizing — most commonly, `JWT_SECRET` couldn't be resolved from Secrets Manager (check the execution role's `secretsmanager:GetSecretValue` permission on the right ARN) or a health-check path mismatch.
- CodePipeline Source stage stuck "pending" — the CodeStar Connection status must be **Available**, not **Pending**; Terraform can create the connection resource but cannot complete the interactive GitHub authorization for you.

19.8 Rotating the JWT secret manually

```
aws secretsmanager put-secret-value \
  --secret-id <project>-<environment>-jwt-secret \
  --secret-string "$(openssl rand -base64 48)"
aws ecs update-service --cluster <cluster> --service <service> --force-new-deployment
```

Rotating invalidates all previously issued JWTs by design, since they were signed with the old secret. This project does not wire up automatic Secrets Manager rotation — that is listed honestly as a future improvement (Section 21), not an implemented feature.

20 Limitations

Stated directly and without hedging, because an honest limitations list is more credible in a viva than a project that claims to have none:

- **No live AWS deployment.** Terraform is verified as internally consistent (`fmt` , `validate` , a full `plan` graph build) but has never been `apply` 'd against a real AWS account.
- **In-memory user store.** `userStore.js` is a `Map` , not a database; data does not survive a process restart, and the app cannot scale horizontally with consistent user data as implemented.
- **No DAST.** All scanning is static/build-time (SAST, SCA, secrets, IaC, container image) — nothing exercises the running service dynamically.
- **No image signing or SBOM-as-gate-input.** Images are digest-pinned but not cryptographically signed; Syft's SBOM output is generated but currently informational, not yet compared against a policy.
- **Single NAT gateway.** A deliberate cost/availability tradeoff for a student account, not a highly-available (per-AZ) NAT configuration.
- **No CloudTrail / AWS Config.** Kept out of scope to keep AWS cost minimal for this demo; it is the single largest documented gap versus a production-grade deployment (per the threat model's own assessment).
- **No automated secret rotation.** `JWT_SECRET` rotation is a manual, documented procedure (Section 19.8), not a scheduled Lambda.
- **No frontend.** Deliberate scope decision (Section 8), not an unfinished feature.
- **No machine learning.** Deliberate design choice (Section 11) — a deterministic policy engine is the correct fit for this decision.
- **No production TLS on the ALB.** No ACM certificate/Route 53 hosted zone is provisioned by default; the ALB uses plain HTTP on port 80 to avoid the setup cost/friction of a verified custom domain for a demo deployment.

21 Future Enhancements

1. **Complete the real AWS deployment.** Run `terraform apply` against an actual AWS account, complete the CodeStar GitHub authorization, and trigger a real pipeline execution end to end — the natural next step, since Terraform is already verified as ready.
2. **Run the `docs/research-notes.md` methodology for real.** Execute the Gated-vs-Advisory experimental design against a corpus of clean and deliberately-vulnerable commits to actually measure Deployment Block Rate, Vulnerability Detection Rate, False Positive Rate, and the other defined metrics — turning the stated hypotheses (H1–H3) into measured findings.
3. **DAST via OWASP ZAP** against the live deployed endpoint, added as a further pipeline stage or a scheduled scan.
4. **SBOM as a gate input**, not just informational output — comparing Syft's CycloneDX/SPDX output against a policy (e.g. "no new unreviewed component") rather than only generating it.
5. **Image signing / SLSA provenance** via Sigstore/cosign, closing the most significant documented supply-chain gap.
6. **CloudTrail + AWS Config** for account-level audit trail and configuration drift detection.
7. **RDS or DynamoDB-backed user store**, replacing the in-memory `Map` behind the same narrow repository interface (`findByEmail` , `create`) it already exposes, so the controller layer needs no changes.
8. **Automated secret rotation** for `JWT_SECRET` via a scheduled Secrets Manager rotation Lambda.

22 Resume Explanation

Three first-person explanations at different lengths, for different interview moments — every claim in them is something demonstrated earlier in this guide.

22.1 30-second version

"I built a CI/CD pipeline for a small Node.js API where deployment is automatically blocked unless five different security scans — code, dependencies, secrets, infrastructure code, and the container image — all pass a written policy. The key design decision is that it fails closed: if a scan doesn't run at all, that counts as a failure, not a pass, so there's no way to silently ship something unverified. I proved that with a 7-scenario automated test suite against the gate logic itself, and I built the whole AWS side — IAM, networking, ECS — with least-privilege roles for every actor in the pipeline."

22.2 1-minute version

"The core problem I wanted to solve was that most teams run security scanners in CI but treat them as advisory — a report gets generated, and the deploy happens anyway unless a human notices and stops it. I built a policy engine, `security-gate.sh`, that makes this deterministic instead: it reads normalized JSON reports from five scanners — Semgrep for code, npm audit for dependencies, Gitleaks for secrets, Checkov for Terraform, Trivy for the container image — compares them against thresholds in a YAML policy file, and its exit code is the single thing the CodeBuild pipeline depends on to decide whether to push the image and update the ECS service. The part I'm most confident explaining is the fail-closed logic: a missing report, a malformed report, or a scanner that crashed is treated exactly like a real policy violation, not skipped — and I proved that's actually true with an isolated fixture test suite, seven scenarios, all passing, independent of whether any real scanner is even installed. On the infrastructure side, I built this in Terraform with four separate least-privilege IAM roles instead of one broad role, including a privilege-escalation guardrail on `iam:PassRole` that a lot of pipelines get wrong."

22.3 2-minute technical version

"This project is a DevSecOps CI/CD pipeline, not a frontend or ML project — the actual product is the enforcement mechanism, and a small Express API exists mainly to give that pipeline something real to build and deploy. The pipeline is GitHub → CodePipeline → CodeBuild → ECR → ECS Fargate, with no manual approval stage in CodePipeline at all — the enforcement point is entirely inside CodeBuild's build phase. Five scanner wrapper scripts each normalize a different tool's output into one shared JSON shape: scanner name, status, and severity counts. That normalization is what lets one gate script, `security-gate.sh`, evaluate all five categories the same way, and it's also what let me test the gate's decision logic completely in isolation — I wrote seven fixture report sets covering a clean pass, a threshold violation, a zero-tolerance secret finding, and four different ways a scan can fail to produce trustworthy evidence — missing file, malformed JSON, a crashed scanner, and a missing tool — and ran the real gate script against all of them. All seven pass, which is the thing I'm proudest of technically, because it means the fail-closed property isn't just a design intention, it's a tested, provable property of the code. On the IAM side, I split permissions across four roles — the pipeline role, the CodeBuild role, the ECS task execution role, and the ECS task role — and specifically separated execution-role

permissions, which the ECS agent uses to pull the image and resolve secrets, from task-role permissions, which the running application itself gets, so that even a full compromise of the running app can't reach the broader pull/secrets permissions. I also resolve the pushed image to its content digest before writing it into the deploy artifact, instead of trusting a mutable tag, which closes a specific tampering window between the gate's pass decision and ECS actually pulling the image. I haven't deployed this to a live AWS account yet — I verified Terraform is internally consistent and the full resource graph builds correctly with `terraform plan`, but running it against real AWS credentials is the next concrete step."

23 Viva / Interview Preparation

Answers below reference the actual implementation directly — file paths, exact behavior, and verified results — rather than generic textbook answers.

Q. Why did you build this project?

A. Most CI/CD security tooling I'd read about treats scanning as advisory — a report is generated, but nothing in the pipeline's control flow actually depends on it. I wanted to build the specific mechanism that makes enforcement automatic and impossible to silently bypass, and prove that mechanism is correct with tests, not just claim it works.

Q. What problem does this solve?

A. It closes the gap between detection and prevention in CI/CD security scanning. Scanning alone doesn't stop a vulnerable build from shipping if a human has to read a report and choose to act — this project makes the deploy/no-deploy decision a deterministic function of scan evidence, enforced automatically inside the build, with no manual approval step to route around.

Q. Why these specific technologies?

A. Node.js/Express for a fast, realistic API surface to exercise the pipeline; AWS CodePipeline/CodeBuild/ECS because they're a coherent, natively-integrated toolchain (the ECS deploy action consumes CodeBuild's artifact format directly); Terraform for reviewable, versioned infrastructure, which also gives Checkov something concrete to scan; and five purpose-built open-source scanners (Semgrep, Gitleaks, Checkov, Trivy, plus npm audit) rather than one all-in-one tool, because each covers a genuinely different threat category and I wanted the normalization/adaptor pattern to be real, not trivial.

Q. Walk me through the architecture.

A. Developer pushes to GitHub → a CodeStar Connection (tokenless) triggers CodePipeline → the Build stage runs CodeBuild against `ci/buildspec.yml`: install, lint, unit tests, Docker build, five scanners, then `security-gate.sh` — with no error suppression on the gate call. On PASS, the image is pushed to ECR with its digest resolved and pinned into `imagedefinitions.json`; the Deploy stage then updates the ECS Fargate service, which runs in private subnets behind an ALB in public subnets. On FAIL, the Build stage fails and CodePipeline never reaches Deploy.

Q. Walk me through the data flow for a single API request.

A. HTTPS request hits the ALB → forwarded to a healthy ECS task in a private subnet → inside the app: Helmet sets secure headers, the body is parsed with a 10kb limit, pino-http logs the request with sensitive fields redacted, a per-IP rate limiter checks the request, the route's express-validator chain validates input shape, then for protected routes the `authenticate` middleware verifies the JWT and attaches `req.user`, then the controller runs, then a response is returned — any thrown error goes through one centralized handler that never leaks a stack trace to the client.

Q. How does authentication work?

A. Registration hashes the password with bcrypt at 12 salt rounds and issues a JWT signed with a server-side secret, carrying an issuer claim and an expiry. Login re-hashes and compares — and specifically always runs `bcrypt.compare()` against either the real stored hash or a fixed dummy hash even when the user doesn't exist, so a non-existent-email response and a wrong-password response are identical in both timing and message, preventing user enumeration via timing analysis. Protected routes verify the JWT's signature and issuer via a small `authenticate` middleware.

Q. Where's the AI/ML component?

A. There isn't one, deliberately. The gate's job is a binary, auditable, compliance-relevant decision from a handful of numeric inputs — severity counts per category — and a deterministic threshold comparison is strictly the right tool for that: every FAIL traces to one readable rule a security reviewer can verify by reading the policy YAML, the exact same input always produces the exact same output (which is what makes my 7-scenario test suite a complete correctness proof rather than a sample), and there's no training-data dependency or model drift to worry about. If I added ML anywhere, it would be as an anomaly-detection layer on top of this deterministic gate — never a replacement for it.

Q. Why this algorithm/design for the gate specifically?

A. The core design choice is fail-closed semantics: a missing report, a malformed report, or a scanner that crashed is treated exactly like a real policy threshold violation, not skipped. I chose that over the more common fail-open default (where anything ambiguous is treated as "fine, proceed") because a fail-open gate degrades silently — a network blip during a tool install, or one bad scanner version, quietly stops enforcing anything, and nobody notices until an incident. Fail-closed means an incomplete security check can never accidentally look like a passing one.

Q. How does the system handle failure — what happens when a scan can't run?

A. Each scanner wrapper script's own exit code (0/1/2) only communicates whether it produced a trustworthy report — tool not installed writes `status:"tool_missing"`, a crash mid-run writes `status:"error"`. Neither of those is the gate's decision; the gate independently checks report validity — file exists, parses as JSON, status is exactly `"completed"` — and treats any failure of that check as a policy violation by default, controlled by two explicit fail-closed flags in the policy YAML. This is validated directly by four of the seven fixture scenarios in `tests/security/run-tests.sh`.

Q. What is the project's security posture, concretely?

A. Defense in depth across layers I can each point to a file for: bcrypt + constant-shape login at the auth layer; Helmet + rate limiting at the transport/app layer; a redacting structured logger and a stack-trace-free error handler for information disclosure; Secrets Manager/SSM injection at container startup for secrets; four least-privilege IAM roles with every wildcard action documented and conditioned where possible; immutable, scan-on-push ECR plus digest-pinned deploys for supply-chain integrity; and a fail-closed policy engine tying all the build-time scanning together into one enforced decision. What's explicitly not covered: DAST, fuzzing, formal pentesting, a WAF, and automated secret rotation — I list these honestly as gaps rather than claim coverage I don't have.

Q. How would this scale?

A. The ECS service can scale horizontally by increasing `desired_count` and adding an autoscaling policy on CPU/ALB request count — the app itself is stateless except for the in-memory user store, which is exactly why that store is the first thing I'd replace (with DynamoDB or RDS) before scaling past one task, since two tasks today would each have their own inconsistent view of registered users. On the pipeline side, the scan categories run sequentially in one CodeBuild phase today; at a larger scale I'd parallelize independent scanners (SAST, secrets, and IaC don't depend on each other or on the Docker build) to reduce Build stage wall-clock time.

Q. What are the project's limitations?

A. No live AWS deployment yet — Terraform is verified but not applied. In-memory (non-persistent) user store. No DAST, no image signing, no SBOM-as-gate-input yet (Syft output exists but isn't policy-enforced). A single NAT gateway rather than per-AZ. No CloudTrail/Config. No automated secret rotation. No frontend and no ML — both deliberate, not oversights. I list all of these directly in Section 20 rather than let them surface as surprises.

Q. What would you improve first, given more time?

A. Actually running the AWS deployment and executing the research-notes.md experimental methodology for real — right now I have a logically-proven fail-closed mechanism (the fixture suite) but no measured data on how it behaves across real, repeated pipeline executions. That's the highest-value next step because it turns a design claim into an evidence-backed one.

Q. What was the hardest technical part?

A. Getting the fail-closed semantics exactly right, and — more importantly — proving they were right independent of whether any real scanner was installed. It would have been easy to write a gate that "mostly" fails closed but has an edge case (e.g. an empty-but-valid-JSON report, or a scanner that exits 0 while writing a malformed report) that silently passes. Building the fixture-based test harness, with a `SECURITY_GATE_REPORTS_DIR` override so the real gate script can be pointed at hand-authored report files instead of live scanner output, was what let me actually verify that — seven specific scenarios, including three distinct ways evidence can be untrustworthy (missing, malformed, crashed/missing tool), all correctly blocked.

Q. How does this differ from a basic CRUD app?

A. The API is intentionally minimal — four endpoints, an in-memory store — because it is not the product. The actual product is the CI/CD security-gate pipeline: the policy engine, the scanner-normalization pattern, the fail-closed test harness, and the least-privilege IAM design around it. A basic CRUD app is the thing being deployed; this project is about the mechanism that decides whether it's allowed to be.

Q. What would enterprise-scale deployment of this need beyond what's here?

A. CloudTrail and AWS Config for organization-wide audit trail and drift detection; image signing/SLSA provenance so a deploy can verify not just "this digest passed the gate" but "this digest was actually built by our pipeline and nothing else"; SBOM comparison as an actual gate input; automated secret rotation; a real database (RDS/DynamoDB) behind the user store with connection pooling and read replicas as needed; multi-AZ NAT for availability rather than the single cost-optimized gateway here; and almost certainly parallelized scanning and caching (dependency layers, scanner rule downloads) to keep Build stage duration reasonable as the codebase grows.

Q. Why is there no manual approval step before deploy?

A. Because a manual approval step's effectiveness is entirely mediated by whether a human reads the report correctly and clicks the right button, every single time, under time pressure — which isn't a technical control, it's a hope. The automated gate removes that human mediation point entirely: CodePipeline's Deploy stage simply has no artifact to consume unless the Build stage — which only succeeds if the gate exits 0 — completes successfully. The module's own code comment states this is deliberate.